

Environmental Telemetry System

An automated measurement and remote transmission system for environmental data including temperature, humidity, air quality, and light intensity.

Team: M. Amani

Environmental Telemetry System

Automated Real-Time Monitoring & Remote Transmission of Environmental Data



Problem statement



Environmental Monitoring Challenges

Traditional manual data collection methods are time-consuming, error-prone, and lack real-time capabilities. Industries, agriculture, and smart cities require continuous, remote monitoring of environmental conditions.

Key Points

- Delayed response to environmental changes.
- High labour costs for manual readings
- Limited accessibility to remote locations
- Inability to track trends over time

Solution

1. Automatic Real-Time Monitoring

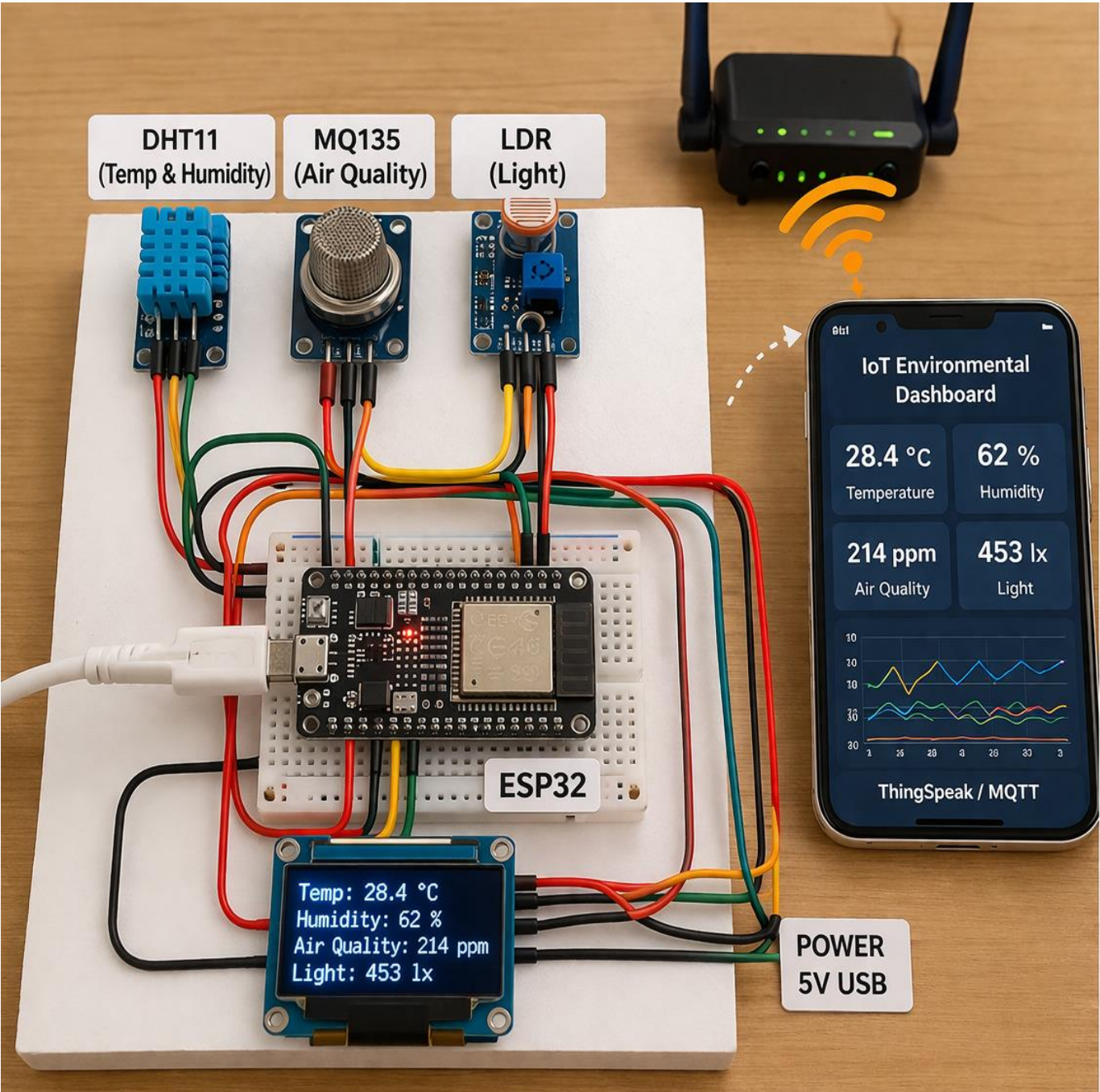
The system continuously collects environmental data (temperature, humidity, air quality, light) using sensors and updates it in real time.

2. Wireless Data Transmission using IoT

Data is sent through the internet using MQTT protocol, allowing fast and reliable communication between the ESP32 and cloud platforms.

3. Remote Access and Alerts

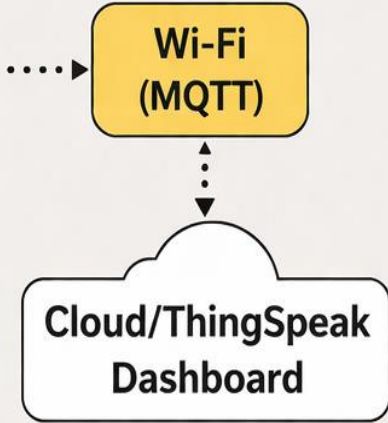
Users can monitor data from anywhere using mobile or computer, and receive alerts when environmental conditions become unsafe.



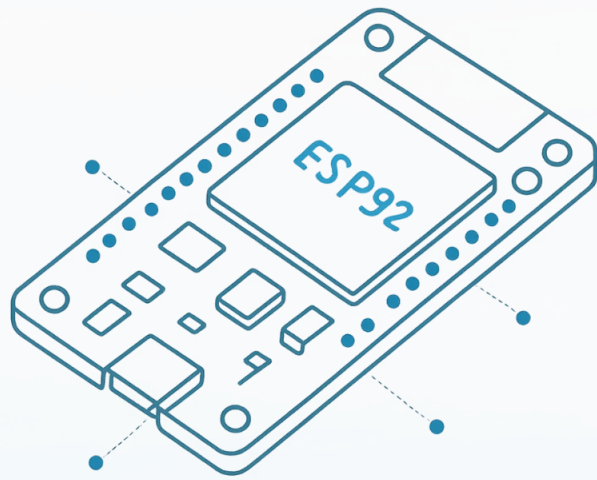
Connections:

ESP32 Pin	Connected To
3V3	DHT11 VCC
GND	All GND
GPIO4	DHT11 DATA
GPIO34	MQ135 AO
GPIO35	LDR AO

- 3.3V (VCC)
- GND
- DHT11 Data
- MQ135 Analog
- LDR Analog

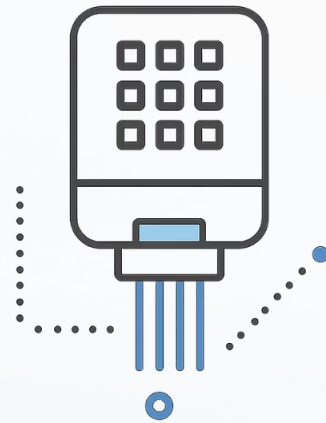


Hardware Components



ESP32 Board

Main controller with built-in Wi-Fi and Bluetooth for wireless communication



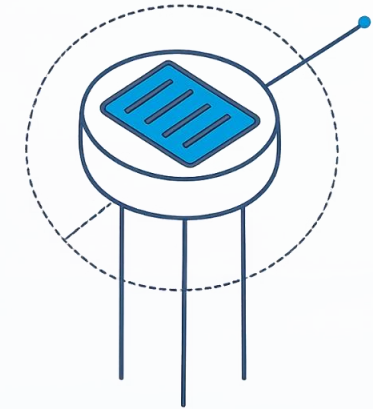
DHT22 Sensor

Measures temperature and humidity with high accuracy



MQ135 Sensor

Detects air quality and harmful gases in the environment



LDR Sensor

Measures light intensity using photoresistive properties

Additional components include breadboard, jumper wires, and USB power supply for complete circuit assembly.

Software Stack

MQTT

Lightweight protocol using publish-subscribe model sends data from ESP32 to server

Wi-Fi

Connects ESP32 to internet. Enables remote monitoring

Security

Protects data. Prevents unauthorised access

Arduino IDE

Primary programming environment for writing and uploading code to ESP32 microcontroller

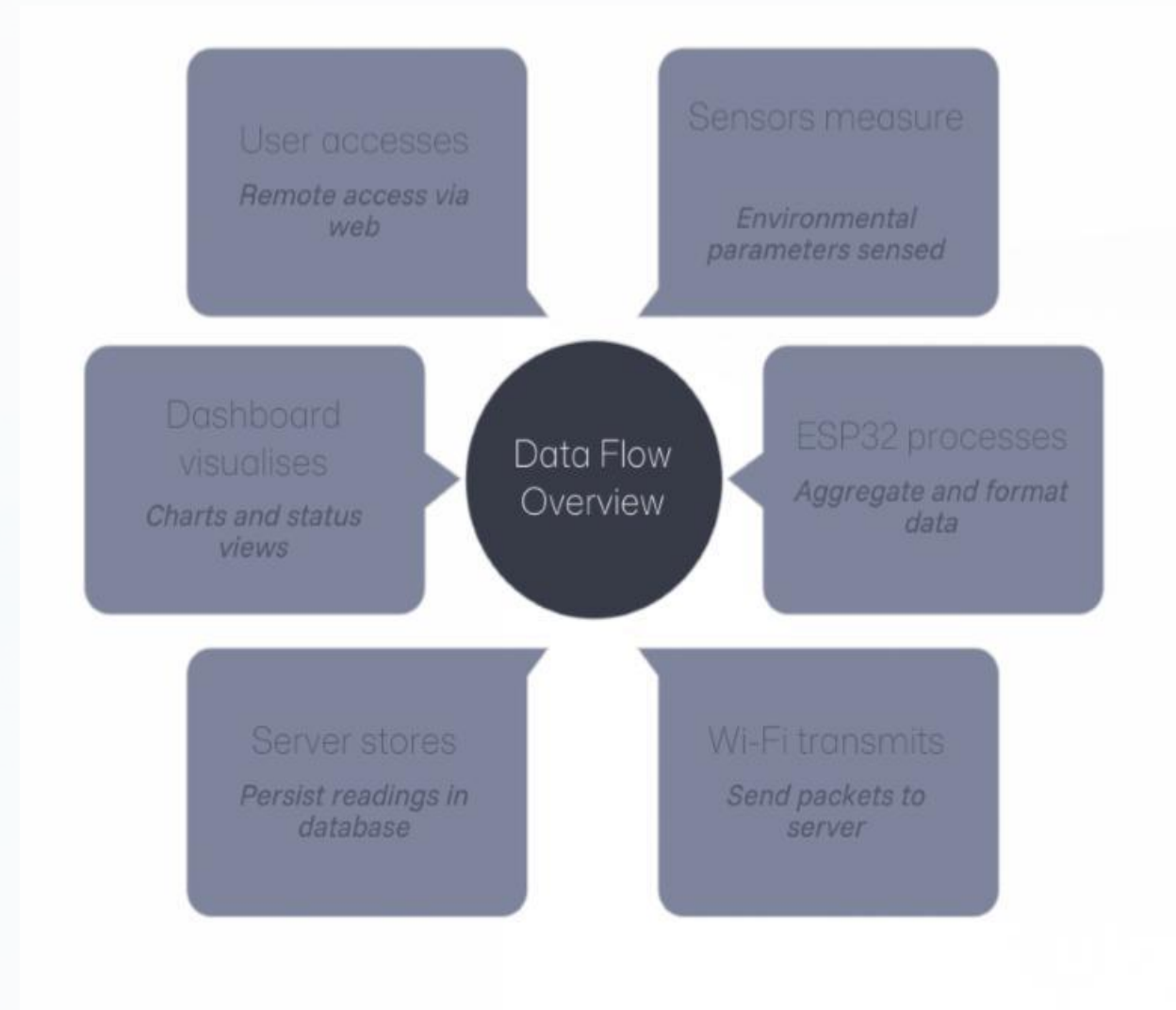
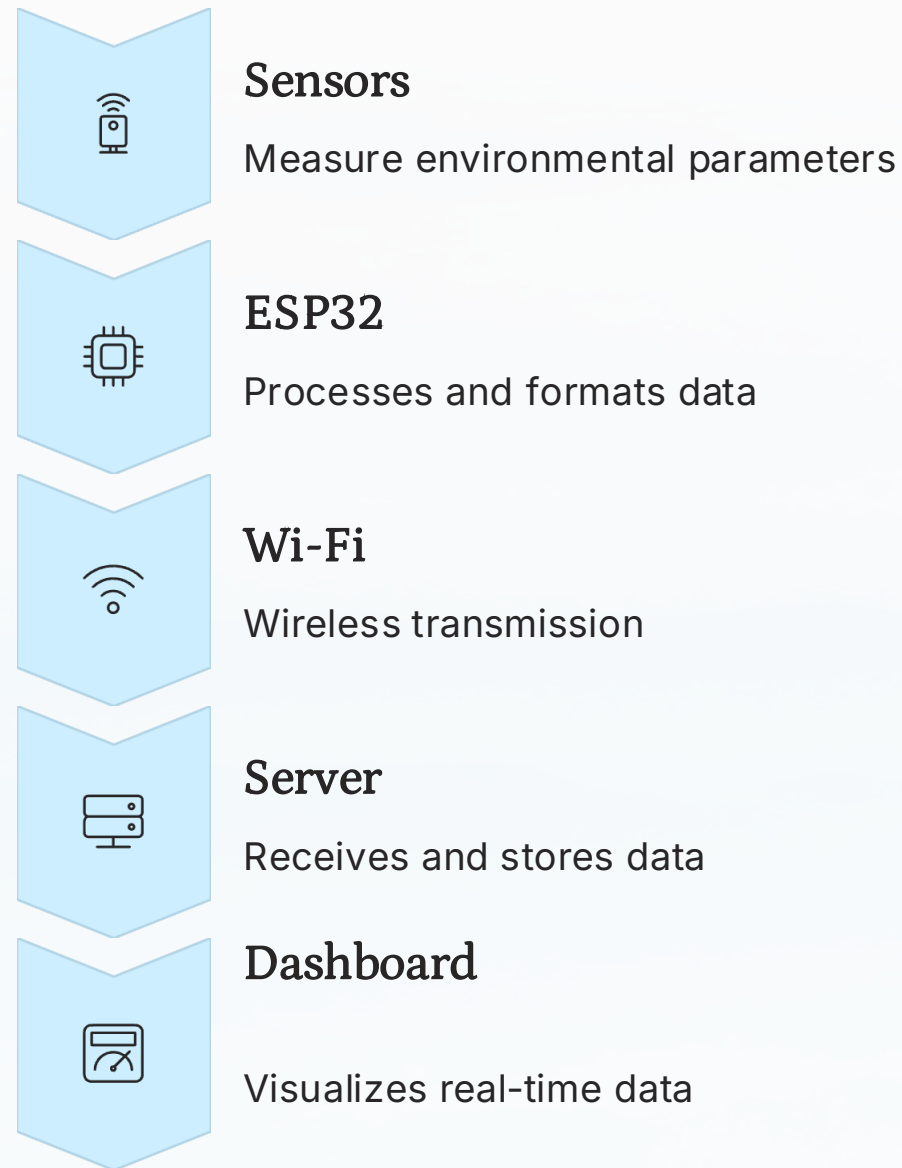
MQTT Broker

HiveMQ or Mosquitto handles lightweight data transmission between ESP32 and server

Dashboard Platforms

Node-RED, ThingSpeak, or Blynk for real-time data visualization and monitoring

System Architecture & Working Principle



The system operates continuously: sensors detect parameters, ESP32 reads through GPIO pins, connects to Wi-Fi, transmits via MQTT protocol, and displays on dashboard in graphs and gauges.

Working of the system

1 Data Collection

- Sensors (DHT11, MIQ35, LDR) collect environmental data



DHT11, MIQ35, LDR, Light

2 Data Processing

- ESP32 reads, processes, and formats sensor data



DHT11, MIQ35, LDR, Light

3 Data Processing

- ESP32 reads, processes, and formats sensor data



DHT11, MIQ35, LDR Light

2 Wi-Fi Connection

- ESP32 reads, processes, and formats sensor data



4 Data Transmission

- ESP32 sends data to cloud via MQTT



5 Data Reception & Processing

- Cloud / IoT platform receives data, processes, and stores it



6 Data Visualization

- Dashboard shows real-time data in graphs and charts



7 User Monitoring

- User accesses dashboard remotely

Sensors → ESP32 → Wi-Fi → MQTT → Cloud → Dashboard → User



Circuit Connections

Power Setup (Common for All)

Arduino 5V → Breadboard + rail

Arduino GND → Breadboard – rail

All components use this power

2. Temperature Sensor (TMP36)

Flat side facing you

Left (VCC) → 5V

Middle (Vout) → A2

Right (GND) → GND

3. Gas Sensor (6-Pin MQ Sensor)

Heater:

H1 → 5V

H2 → GND

Signal:

Any A pin → 5V

Any B pin → A0

Important:

10kΩ resistor between A0 and GND

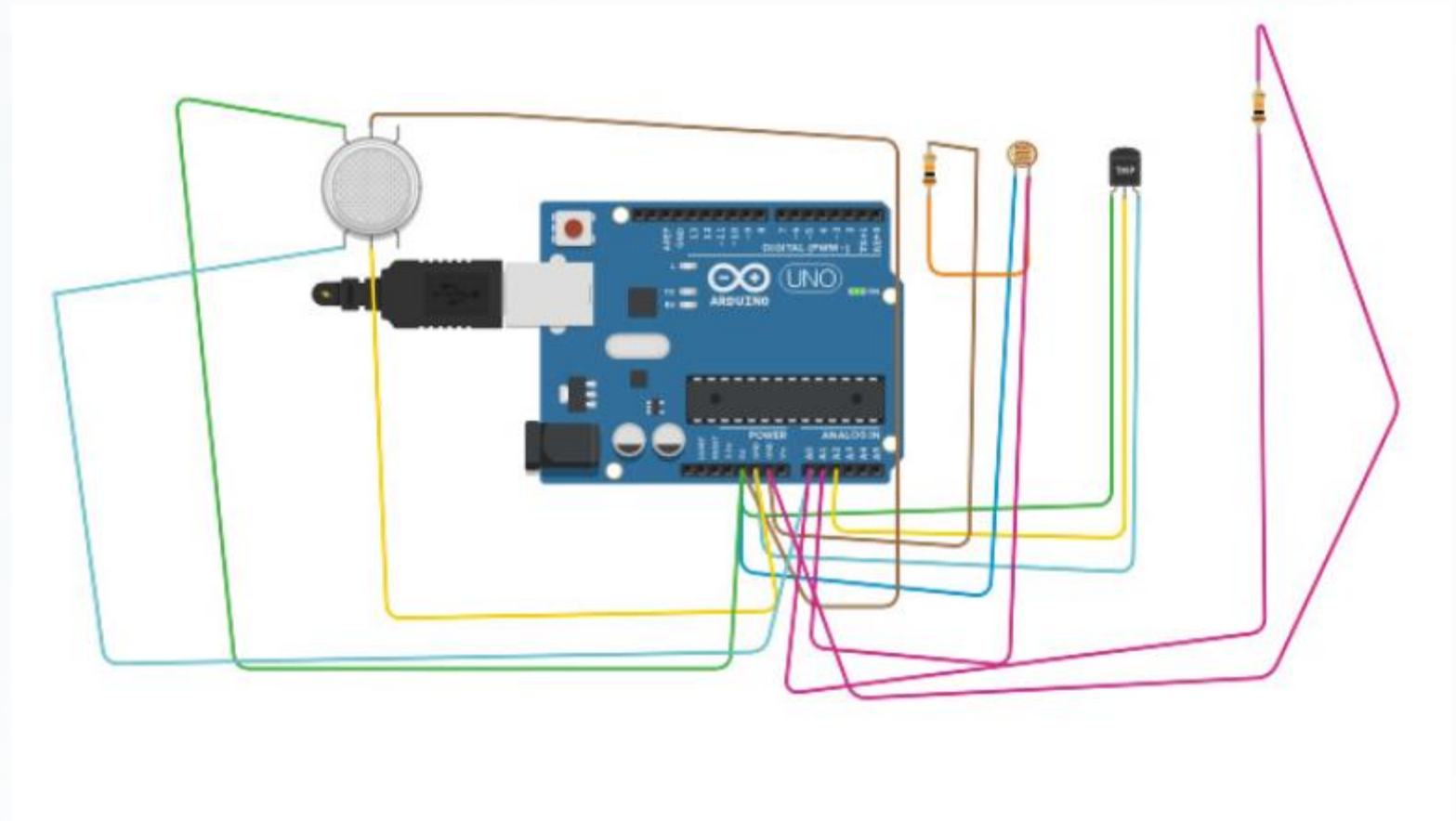
4. LDR (Photoresistor)

Use voltage divider

One side → 5V

Other side → A1

10kΩ resistor from A1 → GND

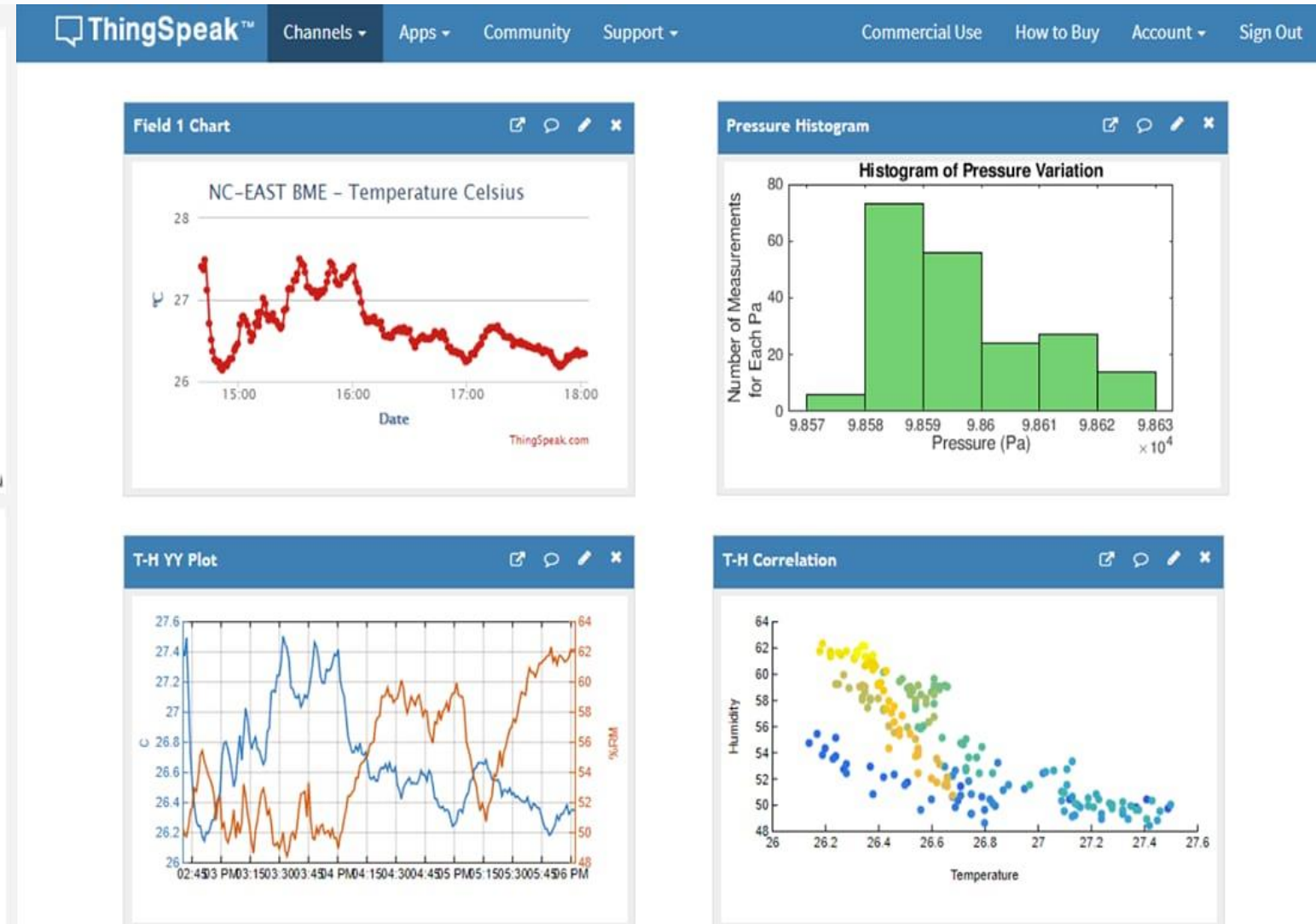
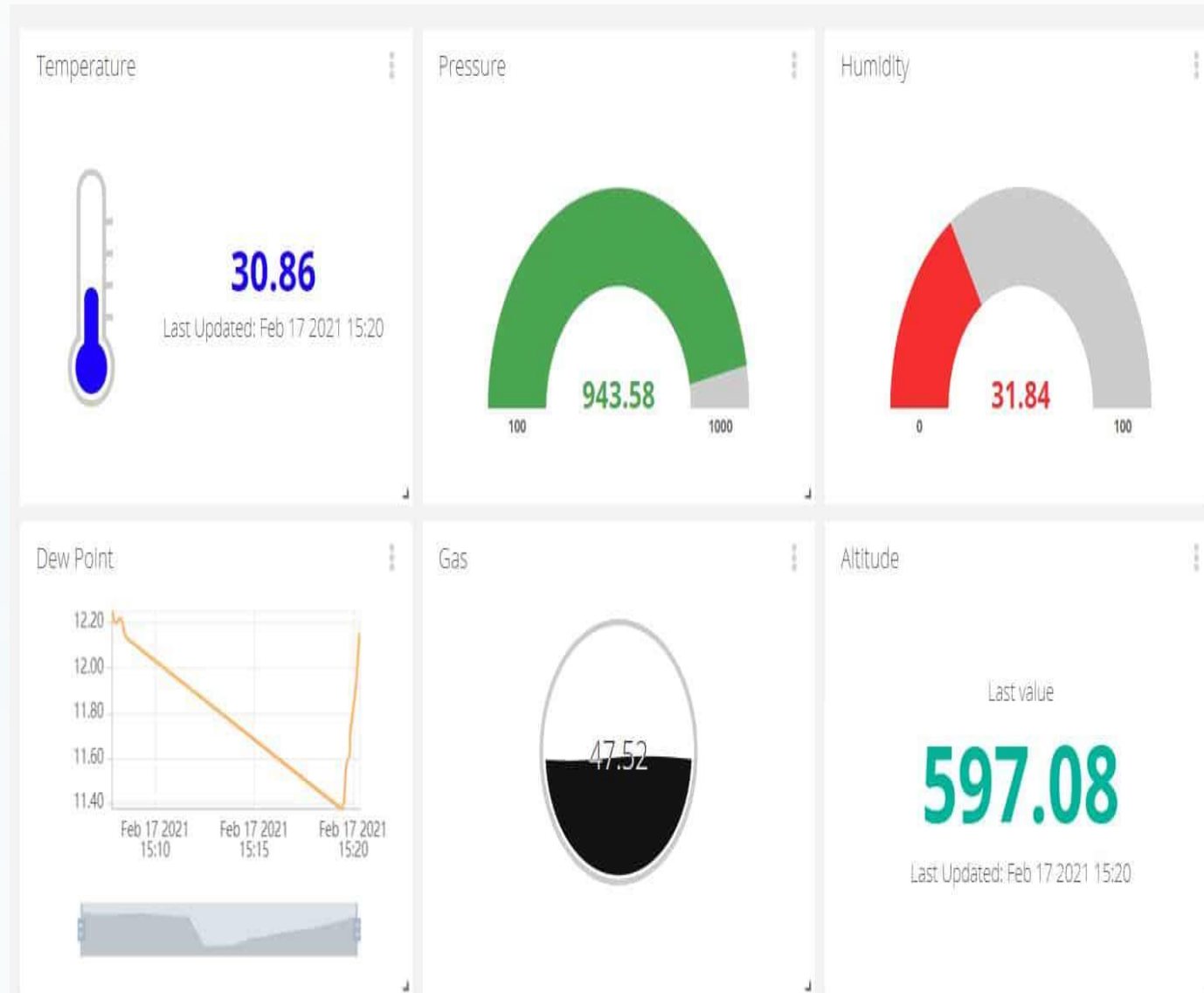


Input code

```
// Pin Definitions
#define TEMP_PIN A2
// Temperature sensor (TMP36 or similar)
#define GAS_PIN A0 // MQ Gas Sensor
#define LDR_PIN A1 // LDR
void setup() {
  Serial.begin(9600);
}
void loop() {
  // Temperature Sensor Reading
  (TMP36)
  int tempValue =
  analogRead(TEMP_PIN);
  float voltage = tempValue * (5.0 / 1023.0);
  float temperatureC = (voltage - 0.5) * 100;
```

```
//Gas Sensor Reading
int gasValue = analogRead(GAS_PIN);
// LDR Reading
int ldrValue = analogRead(LDR_PIN);
// Print Values
Serial.println("----- Sensor Data -----");
Serial.print("Temperature (C): ");
Serial.println(temperatureC);
Serial.print("Gas Value: ");
Serial.println(gasValue);
Serial.print("Light (LDR): ");
Serial.println(ldrValue);
Serial.println("-----");
delay(2000); // wait 2 seconds
```


Dashboard



Applications & Benefits



Smart Homes

Automated temperature and humidity control for optimal comfort



Agriculture

Crop monitoring and environmental optimization for better yields



Industrial Safety

Gas detection and air quality monitoring in hazardous environments



Pollution Monitoring

Real-time air quality tracking for environmental protection



Weather Stations

Accurate environmental data collection for meteorological purposes



Smart Cities

Integrated urban environmental monitoring systems

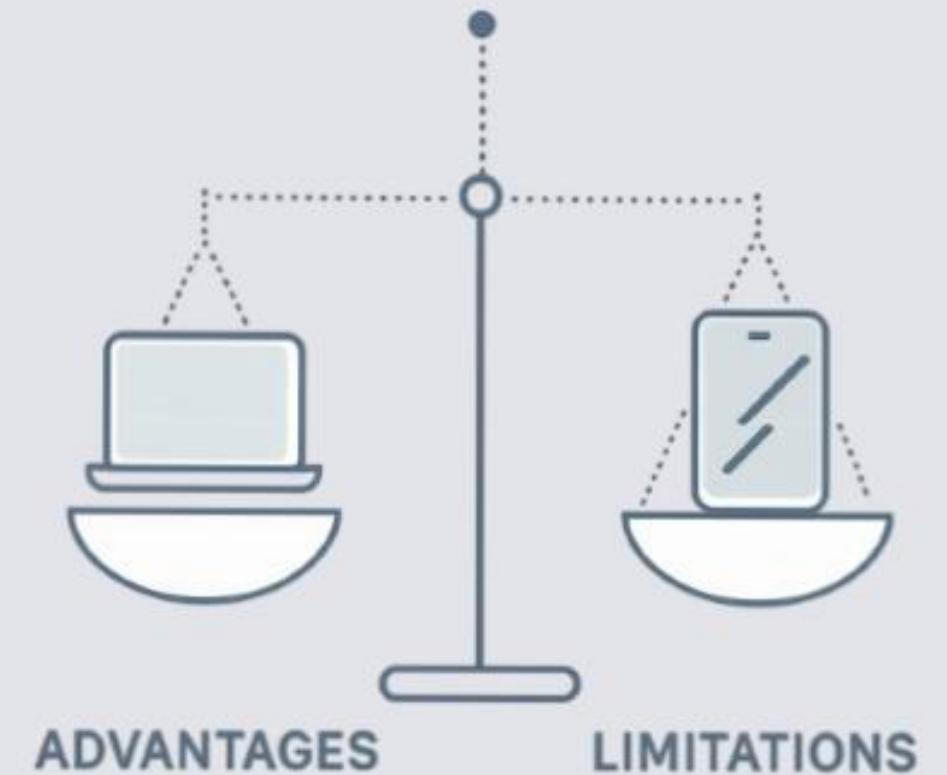
Advantages & Limitations

Advantages

- Real-time monitoring capabilities
- Remote accessibility from anywhere
- Low power consumption design
- Wireless communication eliminates cables
- Scalable and flexible architecture

Limitations

- Requires stable internet connectivity
- Sensor accuracy limitations exist
- Security risks without encryption
- Power dependency on external supply



Future Scope & Testing



AI Integration

Prediction algorithms for environmental trends



Secure Communication

Encrypted data transmission protocols



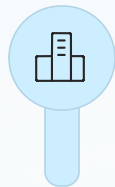
Mobile Alerts

SMS and email notifications for threshold breaches



Solar Power

Sustainable, off-grid deployment options



Smart City Systems

Integration with urban infrastructure networks

Testing Procedure

Connect sensors → Upload code → Connect to Wi-Fi → Start dashboard → Verify readings





Conclusion

The Environmental Telemetry System provides an efficient, cost-effective solution for real-time monitoring and remote data transmission. It forms the foundation for modern smart monitoring systems used across industries, agriculture, and smart cities.

24/7

Continuous Monitoring

Round-the-clock environmental tracking

100%

Wireless

Complete wireless communication

3+

Sensor Types

Temperature, humidity, gas, light



Thank you